

Titanic Survival Prediction-A Statistics and Machine learning Approach

Introduction

In this project I will be using the Titanic Dataset which is most iconic and beginner friendly dataset available in kaggle to build train and test classification models in an data analyst approach, I aim to predict the survival of passengers on the RMS Titanic(**Royal Mail Ship** — a title given to ships that carried mail for the British Royal Mail.) using Supervised machine learning algorithms

This Dataset is partially Real and partially re-constructed ,the **names, age, gender, ticket class, and survival status** are historically accurate for many passengers.

- However, this dataset is a **simplified version created by researchers and Kaggle** for educational purposes.
- It includes only around **891 passengers** from over 2,200 on board, and **some data was filled in or cleaned up** for ease of use.
- It's great for learning, but not 100% of the original manifest.

1. Data Understanding and Cleaning

I have Used 2 CSV Files Train.csv and Test.csv .The Link for this dataset is mentioned below:

Source: Kaggle Titanic Dataset

<https://www.kaggle.com/competitions/titanic/data?select=test.csv>

a)First look at Data:

Train.head()

Test.head()

This Code describes the name of each columns and the number of columns available, `.head()` describes first 5 rows in the dataset

```
train = pd.read_csv(r"C:\Users\user\Desktop\New folder\train.csv")
train.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
test = pd.read_csv(r"C:\Users\user\Desktop\New folder\test.csv")
test.head()
```

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

Next to understand the data types of each column I have used

`train.info()`

`test.info()`

```

train.info()
print("Train info:\n")

test.info()
print("Test info:\n")

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
Train info:

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  418 non-null    int64
1   Pclass       418 non-null    int64
2   Name         418 non-null    object
3   Sex          418 non-null    object
4   Age          332 non-null    float64
5   SibSp        418 non-null    int64
6   Parch        418 non-null    int64
7   Ticket       418 non-null    object
8   Fare         417 non-null    float64
9   Cabin        91 non-null     object
10  Embarked     418 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 36.1+ KB
Test info:

```

Next I will be using

train.describe()

test.describe()

to describe the dataset through mean,count,std e.t.c

```
[29]: train.describe()
```

[29]:	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
: test.describe()
```

:	PassengerId	Pclass	Age	SibSp	Parch	Fare
count	418.000000	418.000000	332.000000	418.000000	418.000000	417.000000
mean	1100.500000	2.265550	30.272590	0.447368	0.392344	35.627188
std	120.810458	0.841838	14.181209	0.896760	0.981429	55.907576
min	892.000000	1.000000	0.170000	0.000000	0.000000	0.000000
25%	996.250000	1.000000	21.000000	0.000000	0.000000	7.895800
50%	1100.500000	3.000000	27.000000	0.000000	0.000000	14.454200
75%	1204.750000	3.000000	39.000000	1.000000	0.000000	31.500000
max	1309.000000	3.000000	76.000000	8.000000	9.000000	512.329200

Next lets see the meaning of each column

Column	Description
PassengerId	Unique ID per passenger
Survived	0 = No, 1 = Yes (Target variable)
Pclass	Ticket class (1 = 1st, 2 = 2nd, 3 = 3rd)
Name	Name of the passenger
Sex	Gender
Age	Age in years
SibSp	# of siblings / spouses aboard
Parch	# of parents / children aboard
Ticket	Ticket number
Fare	Fare paid
Cabin	Cabin number (lots of missing values!)
Embarked	Port of Embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

B) Missing Value analysis

While using `.head()` i could notice some missing values filled as NAN ,computers cannot handle dataset with missing values so in this analysis i will be using the sum query to find the number of missing values in each column

```
print("\nMissing values in Train Dataset :\n")
print(train.isnull().sum())

print("\nMissing values in Test Dataset :\n")
print(test.isnull().sum())
```

Missing values in Train Dataset :

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2
dtype: int64	

Missing values in Test Dataset :

PassengerId	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0
Fare	1
Cabin	327
Embarked	0
dtype: int64	

Handling Missing Values

A) Age Column (Numerical variable)

In Train.csv age column has 177 missing values

In Test.csv age column has 86 missing values

so I was unsure that if the age column is normally distributed, so if i had to take mean, the average might not be distributed properly because there may be some passenger who is an infant and some passengers who is more than 70 year old.

so i am filling my values with **median** which is a better choice here because it's **not affected by extreme values** and gives a more **reliable central value**.

```
train.loc[:, "Age"] = train["Age"].fillna(train["Age"].median())
test.loc[:, "Age"] = test["Age"].fillna(test["Age"].median())
```

B) Embarked Column

In this Embarked column i will have to fill the missing values with mode because embarked means the port in which the ship is available or the port in which the passengers are available to get inside the ship .this dataset has 3 ports, Port of Embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

"Only 2 values were missing. I filled them with the most frequent port of embarkation (mode)."

```
train.loc[:, "Embarked"] = train["Embarked"].fillna(train["Embarked"].mode()[0])
```

C) Fare column

There was only 1 missing value so i decided to fill it with median again

```
test.loc[:, "Fare"] = test["Fare"].fillna(test["Fare"].median())
```

D) Cabin Column

There are too many missing values in this column so i have decided i could not fill these and decided to drop the column

"Since more than 75% of values in the 'Cabin' column were missing, I decided to drop it."

```
train.drop('Cabin', axis=1, inplace=True)
test.drop('Cabin', axis=1, inplace=True)
```

Exploratory Data Analysis (EDA)

Under this section I would like to visually explore patterns and relationships in the dataset using matplotlib library. Specifically this section will contain how survival rates differ based on **Sex, Pclass, Age, SibSp, Parch, and Embarked**.

Step 1: Calculate survival rate by gender

```
[9]: gender_survival = train.groupby("Sex")["Survived"].mean()
print(gender_survival)
```

```
Sex
female    0.742038
male      0.188908
Name: Survived, dtype: float64
```

so as per the data out of 891 passengers

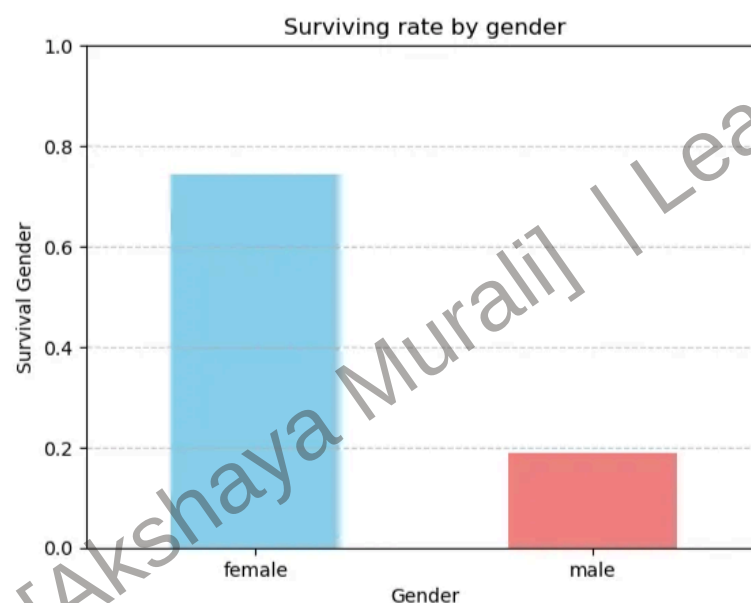
74.2% of Females survived

18.9% of males survived

so lets visualize this percentage using **Bar chart**

```
#Create a bar chart
gender_survival.plot(kind='bar', color=['skyblue', 'lightcoral'])

# Plotting the chart
plt.title("Surviving rate by gender")
plt.xlabel("Gender")
plt.ylabel("Survival Gender")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Step 2 : Calculate Survival rate by Passenger Class(Pclass)

This helps us understand survival rates by passenger class

Here's how to do it using **Matplotlib**:

```
Pclass_survived = train.groupby("Pclass")["Survived"].mean()
print(Pclass_survived)
```

```
Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

so yes according to the sample data and as per google searches, the first class passengers were given access to more lifeboats and thus among 891 passengers

62.9% of First Class passengers survived

47.8% of second class passengers survived and

24.2% of third class class passengers survived and lived to tell the tale

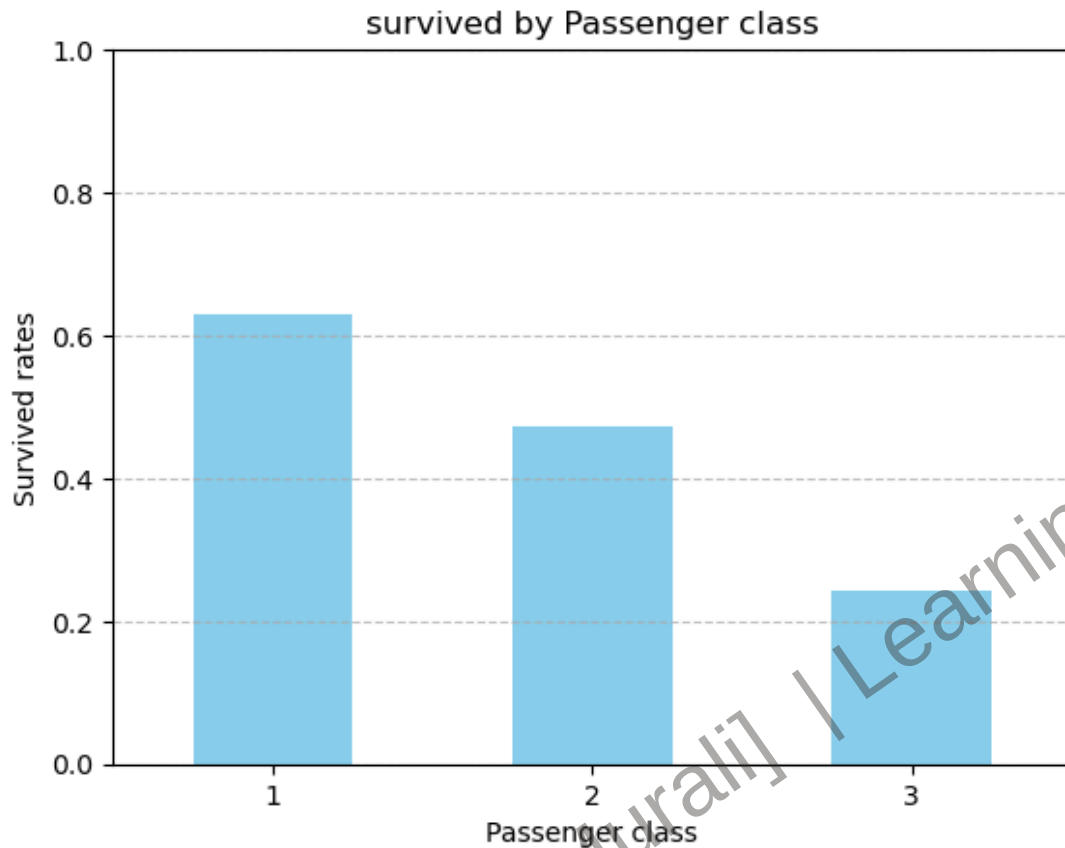
Insight: Survival Rate by Passenger Class

"The survival rate clearly decreases with lower passenger class. First class passengers had the highest chance of survival (over 60%), while third class passengers had the lowest (around 25%).

The saddest part is that **socio-economic status likely influenced access to lifeboats and rescue priorities back in 1915** during the Titanic disaster."

```
# Creating a bar chart
Pclass_survived.plot(kind='bar', color=['skyblue'])

#plot the chart
plt.title("Survived by Passenger class")
plt.xlabel("Passenger class")
plt.ylabel("Survived rates")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Step-3 : Survival Rate by Embarkation Port

Why this chart matters is because i will be able to see the socio economic status of the survived through the region which is the port of embarkation

```
Embarked_survival = train.groupby('Embarked')['Survived'].mean()
print(Embarked_survival)
```

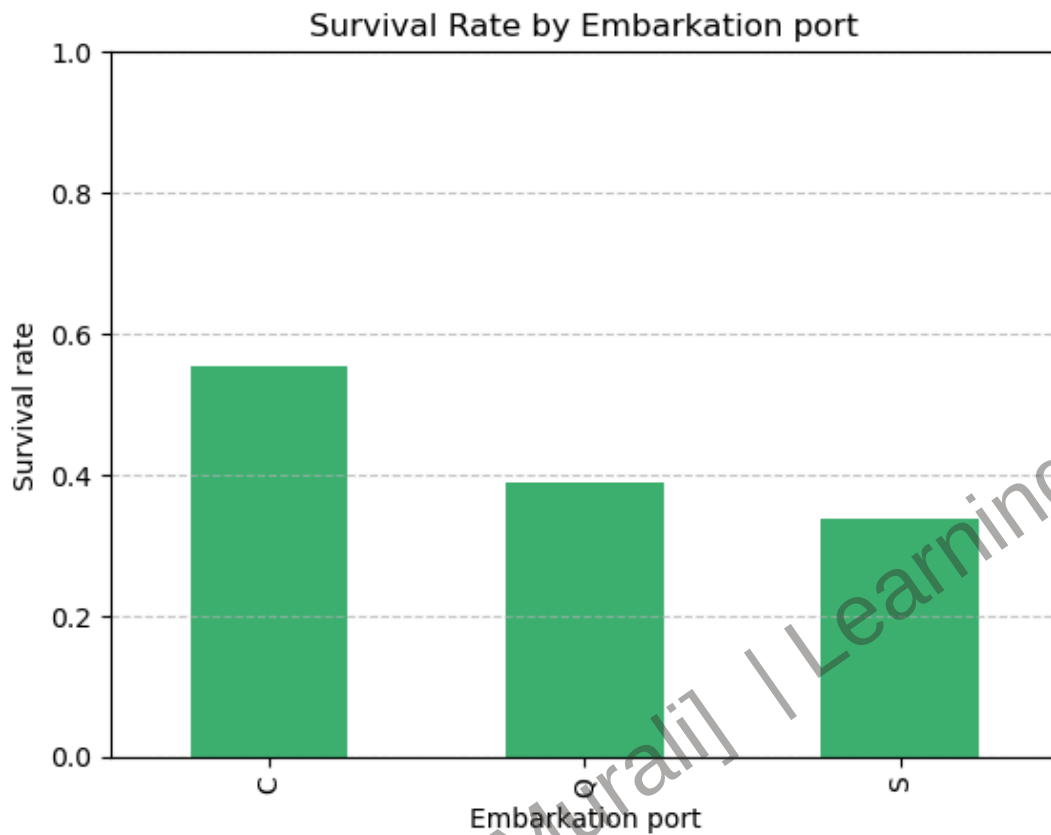
```
Embarked
C    0.553571
Q    0.389610
S    0.336957
Name: Survived, dtype: float64
```

```
#Plotting the chart
Embarked_survival.plot(kind='bar', color=['mediumseagreen'])

#Bar chart
plt.title('Survival Rate by Embarkation port')
plt.xlabel('Embarkation port')
plt.ylabel('Survival rate')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Port of Embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

```
plt.show()
```



```
: pd.crosstab(train['Embarked'], train['Pclass'], normalize='index') * 100
```

```
:      Pclass      1      2      3
Embarked
C  50.595238  10.119048  39.285714
Q   2.597403   3.896104  93.506494
S  19.720497  25.465839  54.813665
```

"Cherbourg was a port city in France known for transatlantic connections and luxury travelers.

and it is clearly shown that half of the passengers who boarded at Cherbourg were first class passengers

The survival rate shows 55.35% survived from Cherbourg

38.9% from Queenstown and

33.6% From Southampton

Step-4 Survival Rate By Age Group

This Chart will help us understand how many percentage of children survived this disaster as the Titanic principle References **“women and children First”**

```
[27]: children = [0, 12, 18, 35, 60, 100]
Sections = ['Child', 'Teen', 'Young Adult', 'Adult', 'Senior']

# Create a new Column in the dataframe
train['AgeGroup'] = pd.cut(train['Age'], bins=children, labels=Sections)

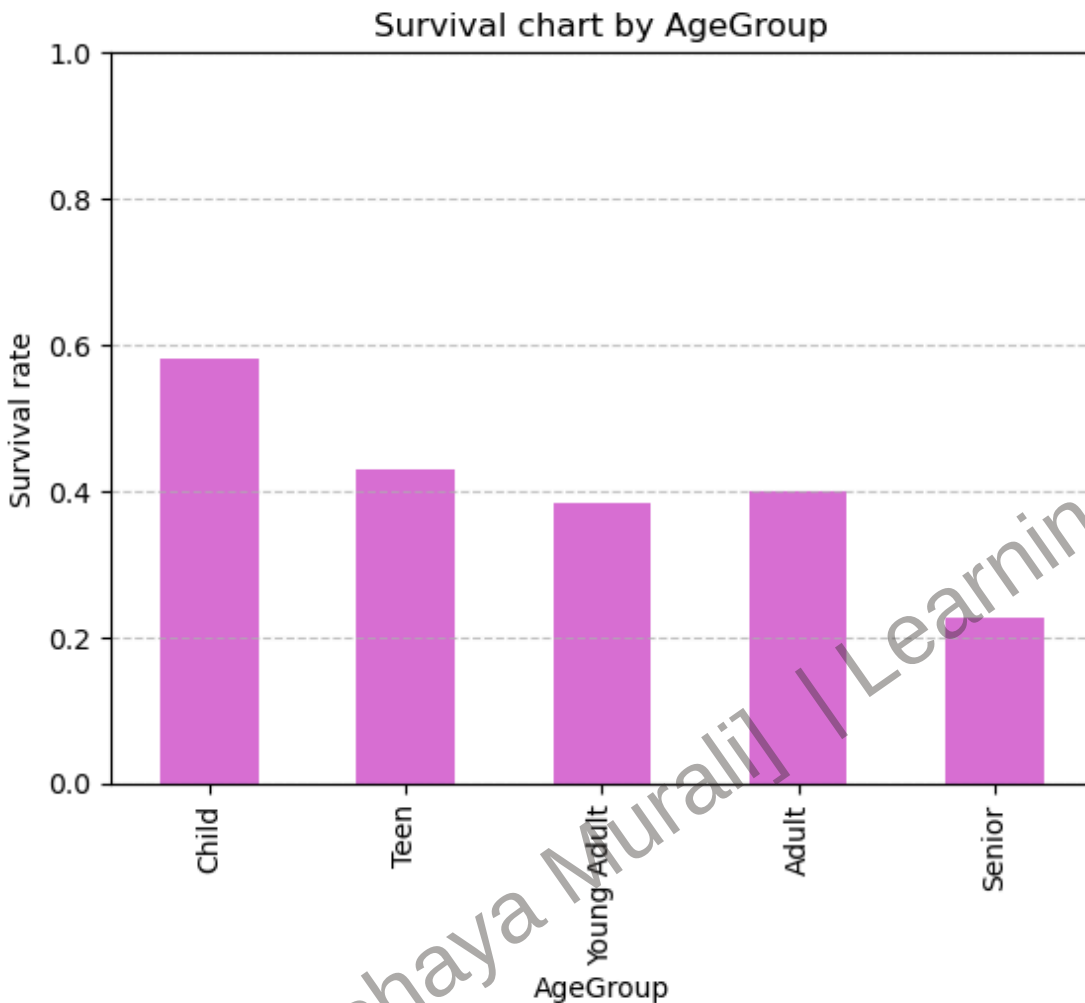
[31]: # Group by age group
age_survival = train.groupby('AgeGroup')['Survived'].mean()
print(age_survival)

AgeGroup
Child      0.579710
Teen       0.428571
Young Adult 0.382682
Adult      0.400000
Senior     0.227273
Name: Survived, dtype: float64

C:\Users\user\AppData\Local\Temp\ipykernel_9460\1144715497.py:2: FutureWarning: The default of observed=False in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the
age_survival = train.groupby('AgeGroup')['Survived'].mean()

[33]: # Plotting the chart
age_survival.plot(kind='bar', color='orchid')

plt.title('Survival chart by AgeGroup')
plt.xlabel('AgeGroup')
plt.ylabel('Survival rate')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Step -5 Survival chart by Number of parents/children aboard

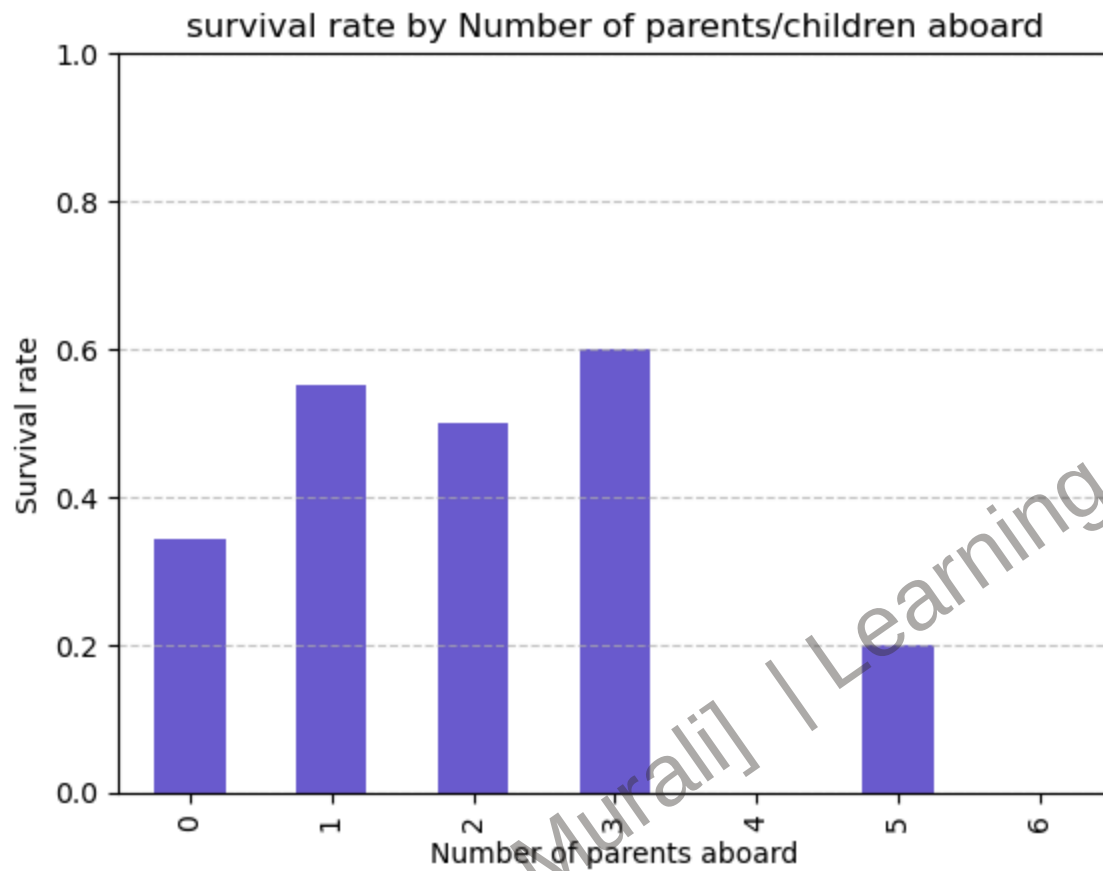
in this section why this chart matters the most is that because i would want to know that if travelling with family means are there more chances of survival? lets answer the Question

```
# Grouping by 'Parch' to get survival rate
parch_survival = train.groupby('Parch')['Survived'].mean()
print(parch_survival)
```

```
Parch
0    0.343658
1    0.550847
2    0.500000
3    0.600000
4    0.000000
5    0.200000
6    0.000000
Name: Survived, dtype: float64
```

```
#Plotting the chart
parch_survival.plot(kind='bar', color= 'slateblue')

plt.title('survival rate by Number of parents/children aboard')
plt.xlabel('Number of parents aboard')
plt.ylabel('Survival rate')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



what this Chart reveals?

Parch	Survival Rate	Meaning
0	~34%	People who traveled alone had a lower chance of survival.
1	~55%	Traveling with 1 parent/child increased the chance of survival.
2	~50%	2 parents/children also had a good survival rate.
3 & 4	60%	Still decent, though data count may be small here.
5 & 6	0-20%	Very low survival — possibly due to large groups being harder to rescue.

OUTLIER DETECTION

what are outliers?

Outliers are extreme values which is extremely low or extremely High, abnormally affecting rest of the values in the dataset

For example

According to google search a normal weight of an polar bear weighs 500 pounds if it weighs less than 350 pounds or extremely High as more than 700 pounds it is called as an outlier

in this dataset lets see which column may have an outlier,

Age - There may be an infant or there may be a person whose age is more than 70

Fare - As I already saw in exploratory data analysis ,1st Class Passengers paid High Fare back in 1915 where they have been given immediate access to lifeboats

```
# Calculate IQR for Fare
Q1 = train['Fare'].quantile(0.25)
Q3 = train['Fare'].quantile(0.75)
IQR = Q3 - Q1

#Step-2
Lower_Bound = Q1 - 1.5 * IQR
Upper_Bound = Q3 + 1.5 * IQR

#Step 3
fare_outliers = train[(train['Fare'] < Lower_Bound) | (train['Fare'] > Upper_Bound)]
print(f"Number of outliers : {fare_outliers.shape[0]}")

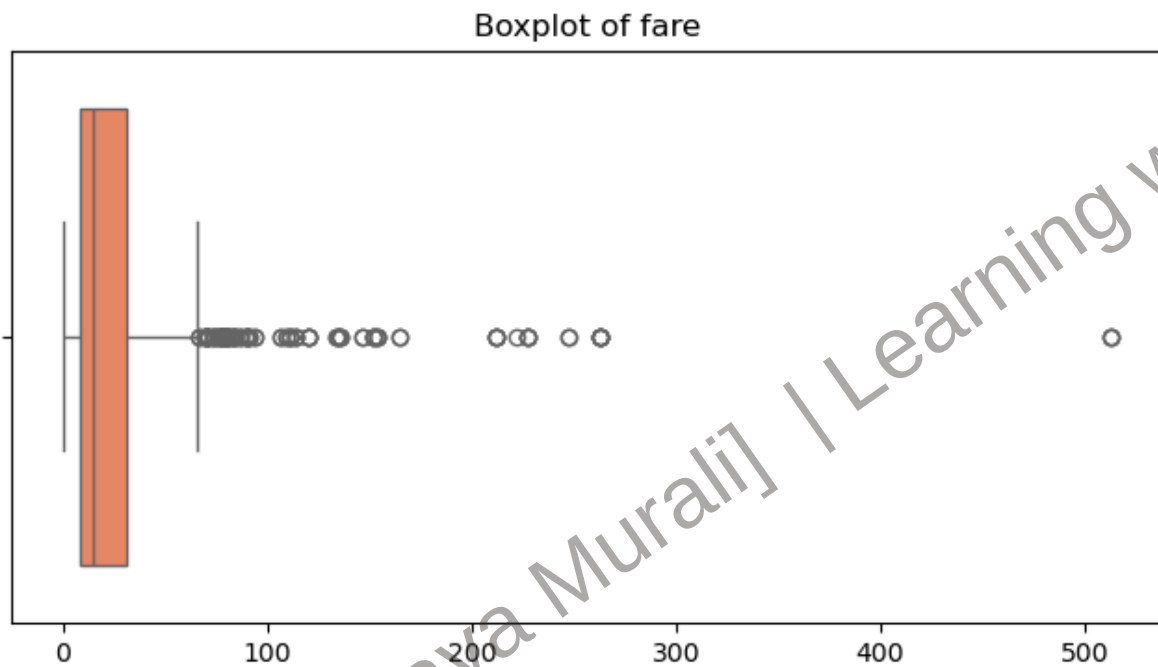
Number of outliers : 116
```

```
low_outliers = train[train["Fare"] < Lower_Bound]
high_outliers = train[train["Fare"] > Upper_Bound]
print(len(low_outliers), "Low Outliers")
print(len(high_outliers), "high Outliers")

0 Low Outliers
116 high Outliers
```

```
import seaborn as sns

plt.figure(figsize=(8, 4))
sns.boxplot(x=train['Fare'], color='coral')
plt.title('Boxplot of fare')
plt.xlabel('Fare')
plt.show()
```



What does the above chart says?

Outlier Count

- **Total Outliers:** 116
- **Low Outliers:** 0
- **High Outliers:** 116

Understanding The Boxplot

The Thick Orange Box(Interquartile range)

Left Edge-25th percentile(Q1)

Right Edge - 75th percentile(Q3)

The middle box contains 50% of the data

(the line inside the box median(Q2)-50th percentile)

upper bound ($Q3 + 1.5 \times IQR$).

lower bound ($Q1 - 1.5 \times IQR$)

The black dots beyond the whiskers are the **outliers** (extreme values).

Summary of the Boxplot

The Data is Right Skewed

Meaning : skewness tells about the asymmetry in the distribution of data where in the titanic dataset most passengers paid low to medium fare prices while few rich passengers paid extremely high VIP Tickets and that's why the data is right skewed

- **There are 116 high outliers (fares above the upper bound).**
- **These outliers represent wealthy or VIP passengers.**

Preparing the Titanic data for Machine Learning(BASICS)

Data preparation for Logistic Regression

Step-1 Removing Un-necessary Columns

In this section i am focusing on survived column as the target column where age,fare,Pclass as the inputs and predicting accuracy_score,confusion matrix and classification Report

we are Dropping Columns like Name ,Cabin ,Passenger Id ,and Ticket ,These inputs are too unique and cannot be converted into numerical variables

```
7]: train_ml = train.drop(columns=['Name', 'PassengerId', 'Ticket', 'Cabin'])
print(train_ml)
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S
..	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S
887	1	1	female	19.0	0	0	30.0000	S
888	0	3	female	NaN	1	2	23.4500	S
889	1	1	male	26.0	0	0	30.0000	C
890	0	3	male	32.0	0	0	7.7500	S

[891 rows x 8 columns]

Step-2 Handling Categorical Variables

Machine Learning models cannot handle Text variables so i am converting 'sex' and 'embarked' column into numeric variable.

```

: # Convert categorical variables to dummy variables
train_ml = pd.get_dummies(train_ml, columns=['Sex', 'Embarked'], drop_first=True)
print(train_ml)

```

	Survived	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q \
0	0	3	22.0	1	0	7.2500	True	False
1	1	1	38.0	1	0	71.2833	False	False
2	1	3	26.0	0	0	7.9250	False	False
3	1	1	35.0	1	0	53.1000	False	False
4	0	3	35.0	0	0	8.0500	True	False
..	...	...	...	...	...	...	...	...
886	0	2	27.0	0	0	13.0000	True	False
887	1	1	19.0	0	0	30.0000	False	False
888	0	3	NaN	1	2	23.4500	False	False
889	1	1	26.0	0	0	30.0000	True	False
890	0	3	32.0	0	0	7.7500	True	True

	Embarked_S
0	True
1	False
2	True
3	True
4	True
..	...
886	True
887	True
888	True
889	False
890	False

Here i learnt that `pd.get_dummies` converts categorical variables into numerical variables and the first port (Cherbourg) is treated as an baseline meaning : if an passenger is boarded through Cherbourg port the `embarked_Q` is shown as false and `embarked_S` is shown as false so the machine learning model automatically learns that the passenger boarded at cherbourg

True/False is an Boolean value it is converted as an numerical value during Training through sci-kit

There are other strong predictors of survival too.

Step-3 Defining Targets and Features

In this step i am featuring X as the input variable and Y as the output variable ,so X is an feature variable so that is where a machine learning model learns from its features and Y is where the model tries to predict the result,so here i am predicting the survival result

```
# define target and features
X = train_ml.drop('Survived', axis=1)
y = train_ml['Survived']
```

.drop represents dropping the survived column from X (axis=1 means columns and axis=0 means rows) because survived column is the target variable

Step -4 Train-Test Split

in this step we will split the data into train and test , 0.2 for test , which is 20% for test and remaining 80% for training , Generally

You **split** the data into:

- X_train , X_test
- y_train , y_test

```
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

the random_state=42 means ensures we get the same kind of split everytime,

Step -5 Initialize and Train the Model

i am importing logistic Regression model using sci-kit learns LogisticRegression() class

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

LogisticRegression

LogisticRegression(max_iter=1000)

so I learnt an real life analogy in relation to the above query to explain the model: lets say we want to park a car accurately ,first time we do not nail in perfection, so we try again and again and the same thing applies to machine learning models each attempt(1 attempt) = 1 iteration and you give the model a maximum of 100 or 1000 times it's like saying to a robot,"Try 1000 times but I want the best prediction or accurate test results"

that's why we use (max_iter=1000) and then the algorithm is also trying to "park" itself into the

best possible fit — meaning, it's adjusting its weights repeatedly to minimize prediction error. X_train ,Y_train are your data features and labels.

Step 6- Make predictions and Evaluate the Training Model

in this step i am asking the model to predict the accuracy score which tells the percentage of overall correct predictions

X_test is given as an input variable so that it predicts the outcome ,Then i can compare the predicted outcome y_pred vs the actual outcome y_test

```
#make predictions
y_pred = model.predict(X_test)

#accuracy Score
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

Accuracy: 0.82
```

1. Output: Accuracy: 0.82 → That means **82% correct predictions!**

Step-7 Confusion matrix

This Step means the same ML model predicts how much percentage of accuracy score it has predicted correctly

The logic I understood behind it is the ML model is split into train and test .test is given an value of 0.2 which is 20% of the data is tested and the ML models is trained with 80% of data

for example

Lets say we have an data of 891 passengers where 891 is split into train and test training is given with 80% of the data $891 * 80/100 = 714$ passengers

i am able to Test the ML model with the remaining 20% $891 * 20/100 = 177$ passengers

This means the ML model learned the patterns with 714 passengers

Then i am asking the ML model to predict the survival outcome by giving the features of X_test

y_pred = ML Model predicts the survival outcome

y_test = This is the actual survival outcome

What is the Confusion Matrix doing Here?

The Confusion Matrix is split into 4 values

True Positives(TP) = Survived and predicted as survived

True Negative(TN) = Did not survive and predicted as not survived

False Positive (FP) = Predicted as survived but did not survive

False Negative(FN) = Predicted as not survived but survived

```
from sklearn.metrics import confusion_matrix

conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)

Confusion Matrix:
[[90 15]
 [18 56]]
```

The Result The ML Model explains is:

True Positives(TP) = 56 - Survived and predicted as survived

True Negatives(TN) = 90 - Did not survive and predicted as not survived

False Positive(FP) = 15- Predicted as survived but did not survive

False Negatives(FN) = 18 - Predicted as not survived but survived

Automated Version

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

def automate_titanic_model(file_path):
    #step 1 : Load Data
    data = pd.read_csv(file_path)

    #step 2 : Handle missing values
    data['Age'] = data['Age'].fillna(data['Age'].median())
    data['Fare'] = data['Fare'].fillna(data['Fare'].median())
    data['Embarked'] = data['Embarked'].fillna(data['Embarked'].mode()[0])

    #step 3 : Encode categorical features
    data = pd.get_dummies(data, columns=['Sex', 'Embarked'], drop_first=True)

    #step 4 : Drop columns not needed
    columns_to_drop = ['Name', 'Ticket', 'Cabin', 'Embarked']
    data = data.drop(columns=columns_to_drop, errors='ignore')

    #step 5 : Define Target and Features
    X = data.drop('Survived', axis=1)
    y = data['Survived']

    #step 6 : Train-Test-Split
    X_test, X_train, y_test, y_train = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#step 7 : Train Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Step 8: Predict & evaluate
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("✅ Model Accuracy:", round(acc * 100, 2), "%")
print("🔗 Confusion Matrix:\n", cm)

return model

model = automate_titanic_model(r"C:\Users\user\Desktop\New folder\statistics+ machine learning project\train.csv") #replace with your csv path
```

```
✅ Model Accuracy: 79.78 %
🔗 Confusion Matrix:
[[381  63]
 [ 81 187]]
```

Second Evaluation: Training & Testing on the Same Data

In my automated workflow function, I trained and tested the model on the **same dataset** (after cleaning), totaling **712 valid passengers** (some rows with missing values were removed).

The model achieved:

- **Accuracy Score:** 79.78%
- **Confusion Matrix Total:** 712 passengers

(because this evaluation used the full cleaned dataset without splitting)

🚩 Note:

This approach **does not reflect real-world testing**, as the model sees the same data it was trained on. It may give a slightly **inflated or biased performance** due to overfitting.

Conclusion

For real-world evaluation, the **first method using train-test split** is more trustworthy.

The second method helped me verify that the model fits the data well, but it shouldn't be used alone to judge performance.

Project with [Akshaya Murali] | Learning with AI